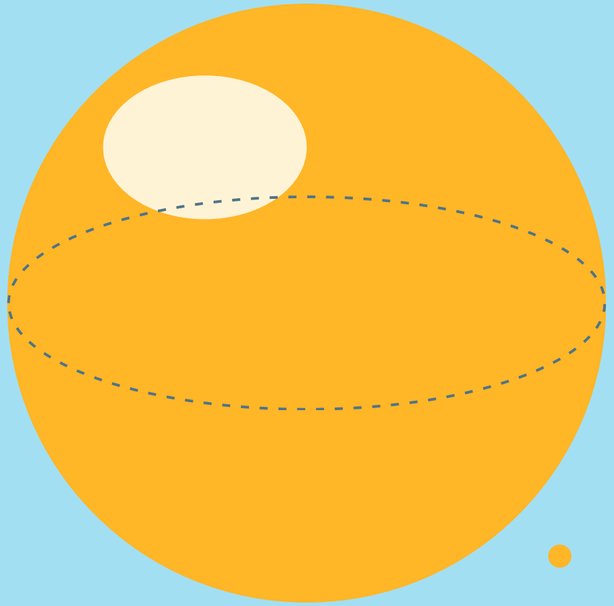
Cannonbolt Rush



Projeto de cenário interativo em 3D desenvolvido com OpenGL

Discentes: Adryele Oliveira
Filipe Corrêa

Docente: Dr. Fabrício Rossy
Disciplina: Computação Gráfica

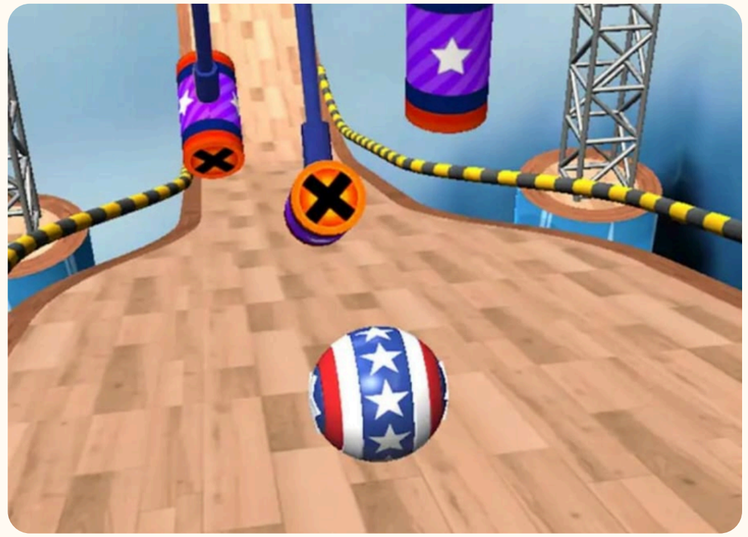
VISÃO GERAL

Conceito proposto

O principal objetivo deste trabalho foi desenvolver um mini jogo 3D utilizando OpenGL em linguagem **C++**, explorando os conceitos estudados na disciplina de Computação Gráfica, inspirado em jogos no estilo **Marble Run e plataformas 3D**.







Nome dos Jogos acima: “Going Balls” e “Super Mario 3D World”.

VISÃO GERAL

Ferramentas e conceitos aplicados

C++ | **OpenGL 3.3 Core** | **GLFW** | **GLAD** | **GLM** | **GLSL**

CONCEITOS DE COMPUTAÇÃO GRÁFICA APLICADOS

- Translação · Rotação · Escala
- Quatérnions
- Coordenadas esféricas
- Iluminação Blinn-Phong
- Texturas procedurais + UV scale
- Shadow mapping
- Z-Buffer
- Back-face culling
- Ray casting
- Colisão com caixa orientada

Nenhuma engine pronta foi usada. Cada técnica acima foi implementada manualmente sobre a API gráfica de baixo nível do OpenGL.

VISÃO GERAL

Um motor de jogo 3D construído do zero

Cannonbolt Rush é um platformer 3D estilo Marble Madness escrito em C++ com OpenGL 3.3 Core, sem nenhuma engine pronta: renderização, física, editor de fases e HUD foram implementados manualmente.

- 3 fases jogáveis com física de plataforma, pulo e checkpoints
- Editor de nível integrado — construção com cliques do mouse
- Câmeras orbitais independentes para editor e jogo
- Iluminação, sombra e texturas 100% geradas em tempo real



Editor de Fases

Colocação, remoção e edição de objetos via raycasting do mouse



Física & Movimento

Gravidade, pulo, colisão e plataformas móveis senoidais



Câmera Orbital Dupla

Coordenadas esféricas (yaw/pitch/distância) por contexto



HUD Próprio

Fonte bitmap 5×7 e widgets desenhados só com retângulos

ARQUITETURA

Pipeline de renderização por quadro

1

Geometria

VAO / VBO / EBO
esfera, cubo, rampa, cilindro

2

Vertex Shader

Model · View · Projection
transforma cada vértice

3

Rasterização

GPU gera fragmentos
a partir dos triângulos

4

Fragment Shader

Iluminação + textura
+ sombra por pixel

5

Framebuffer

Imagem final
na tela (swap buffer)

PASSO EXTRA — SHADOW PASS (executado antes do quadro principal)

A cena é desenhada mais uma vez usando shaderProfundidade a partir do ponto de vista do sol, gravando apenas profundidade em um framebuffer 1024×1024 (depthMapFBO). Resultando no shadowMap, que alimenta o Fragment Shader principal para decidir quais pixels estão na sombra.

shaderProfundidade · shaderPrincipal · shaderHUD

GEOMETRIA · MODEL MATRIX

Transformações geométricas

```
model = translate(pos) × rotate(x,y,z) × scale(escala)
```

Translação

```
glm::translate(model, obj.posicao)
```

Posiciona cada objeto no grid do mundo. No editor, o clique do mouse é arredondado ao tamanho da célula (GRID_TAM) antes de virar posição.

Rotação

```
glm::rotate(model, ângulo, eixo)
```

Aplica-se em X, Y e Z para orientar rampas e obstáculos. A moeda gira continuamente no eixo Y usando o tempo acumulado do jogo.

Escala

```
glm::scale(model, obj.escala)
```

Cada tipo de objeto tem um vec3 de escala próprio — de pisos largos e finos a paredes altas e estreitas.

Além de T·R·S: a rotação de rolagem da bola usa quatérnions (glm::angleAxis) acumulados a cada quadro — evita gimbal lock e faz o giro seguir a direção real do deslocamento.

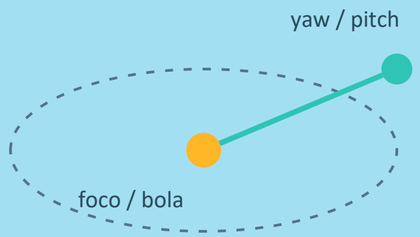
ESPAÇO DE CÂMERA

Configuração da Câmera

Câmera do Editor

Orbita um ponto de foco livre (pan com WASD/QE). Botão direito arrasta para girar yaw e pitch.

```
offset = distância · (cos pitch·sen yaw,  sen pitch,  cos pitch·cos yaw)
```




```
glm::perspective(58°, aspect, 0.1, 260)
glm::lookAt(posCamera, alvo, up)
```

Projeção em perspectiva converte o frustum 3D em coordenadas de recorte; lookAt constrói a matriz de visão a partir de posição, alvo e vetor up.

Câmera no mundo 3D

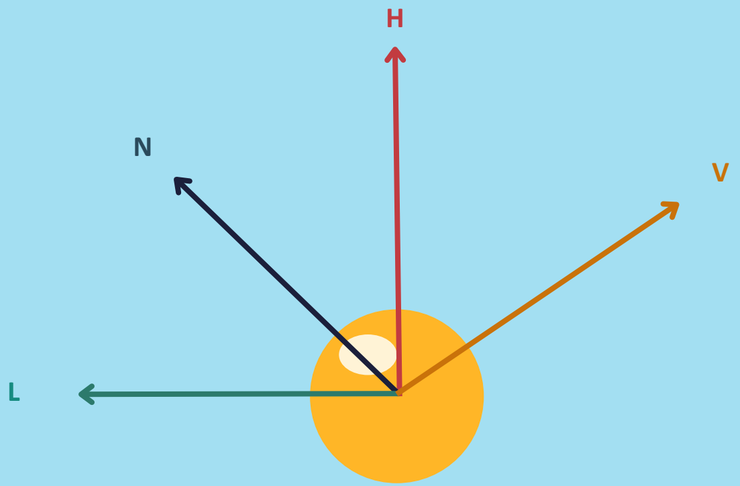
1. Coordenadas de tela → NDC (−1..1)
2. NDC → espaço da câmera (inversa da projeção)
3. Espaço da câmera → mundo (inversa da view)
4. Raio = posCamera + t · direçãoMundo

Esse mesmo raio é reaproveitado em dois lugares do editor:

- Colocar objetos — interseção do raio com o plano da grade (altura Y fixa)
- Selecionar / remover objetos — menor distância entre o raio e o centro de cada objeto

SHADING

Modelo de iluminação: Blinn-Phong



N = normal · L = luz · V = visão · H = normalize(L+V)

```
cor = ambiente + (1 − sombra) · (difusa + especular)
```

Ambiente

`0.48 × cor_base`

Luz de base constante — garante que nada fique 100% preto nas áreas sem luz direta.

Difusa (Lambert)

`max(N·L, 0) × cor_base`

Intensidade proporcional ao ângulo entre a normal da superfície e a direção da luz.

Especular (Blinn)

$0.35 \times (N \cdot H)^{32}$

Brilho concentrado calculado com o vetor halfway — mais barato que o modelo Phong clássico.

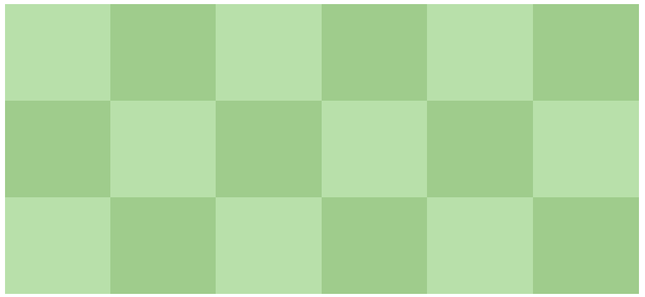
A posição do sol orbita em tempo real (calcularPosicaoSol), então L e as sombras mudam continuamente durante o jogo.

Texturas procedurais



Grade do editor

criarTexturaGrade(): pixels marcados como linha quando (x % passo) ou (y % passo) cai na faixa de espessura.



Piso do jogo (xadrez)

criarTexturaQuadriculada(): alterna entre corA/corB conforme ((x/passo)+(y/passo)) % 2, com mipmaps e wrap repetido.



Bola de canhão, o “Cannonbolt”

criarTexturaCannonbolt(): calotas polares por distância ao polo + listras onduladas com seno, tudo em UV esférico.

Mapeamento UV: TexCoords = aTexCoords × uvScale no vertex shader controla quantas vezes a textura repete sobre a superfície — o chão do jogo usa uvScale de 60× para simular um piso extenso sem aumentar a resolução da imagem.

SHADOW MAPPING

Sombras em duas passadas de renderização

1

Vista da luz

lightView + lightProjection (ortográfica) formam o lightSpaceMatrix, olhando da posição do sol para o alvo da câmera.

2

Depth pass

A cena é desenhada com shaderProfundidade em um framebuffer 1024×1024 — só a profundidade é gravada no depthMap.

3

Comparação

No shader principal, FragPosLightSpace projeta o fragmento no espaço da luz e compara sua profundidade com o depthMap.

4

PCF 3×3

A média de 9 amostras ao redor suaviza a borda da sombra; um bias dinâmico evita o artefato de 'shadow acne'.

```
bias = max(0.006 × (1 − N·L), 0.0015)
```

Só projetam sombra os objetos sólidos, móveis e de repulsão — checkpoints, moedas e a bola ficam de fora do passe de profundidade.

VISIBILIDADE

Algoritmos utilizados



Z-Buffer (pontual)

GL_DEPTH_TEST compara, pixel a pixel, a profundidade do novo fragmento com o valor já armazenado.



Back-face Culling (geométrico)

GL_CULL_FACE descarta triângulos inteiros cuja normal aponta para longe da câmera, usando o sentido de enrolamento (winding order) dos vértices.



Shadow Mapping (pontual, do sol)

Reaplica o mesmo princípio do Z-buffer, mas a partir do ponto de vista da luz: o que 'a luz não vê' fica marcado como sombra.



Ray Casting (geométrico)

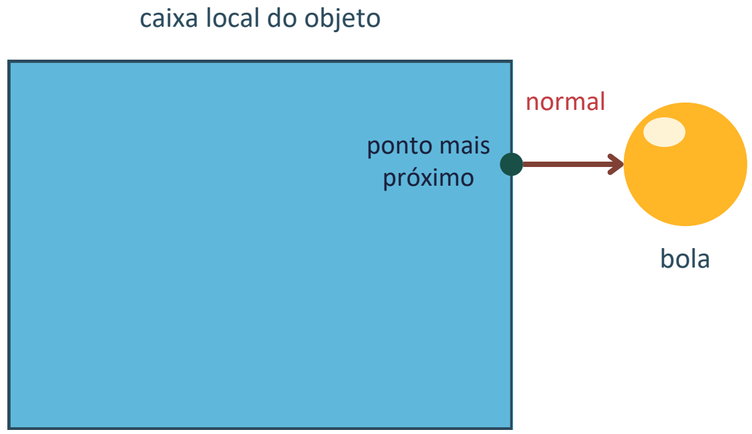
selecionarObjetoPorRaio projeta um raio da câmera e testa a distância até cada objeto que foi usado para colocar, selecionar e remover peças no editor.

GEOMETRIA APLICADA

Física de Colisão

colideComObjeto() resolve a colisão esfera-caixa em 4 passos geométricos:

- Desfaz a rotação do objeto — leva o centro da bola para o espaço local da caixa
- Recorta (clamp) essa posição aos limites ±escala/2 → ponto mais próximo na caixa
- Mede a distância entre a bola e esse ponto; colide se for menor que o raio
- Refaz a rotação para obter a normal de colisão de volta no espaço do mundo

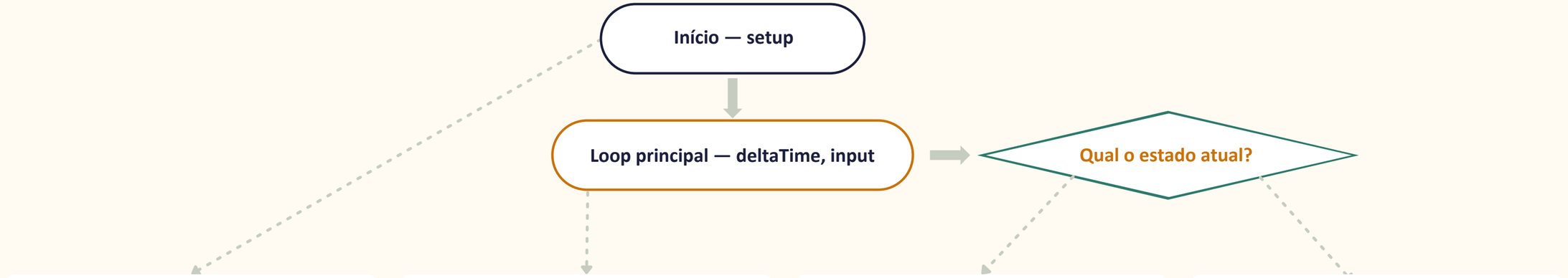



```
posBola = pontoPerto + normal × raioBola
velBola −= normal × dot(velBola, normal)
```

Por que isso importa: como a caixa de colisão gira junto com o objeto, uma rampa rotacionada vira um plano inclinado de verdade — sem essa etapa geométrica extra, a bola atravessaria ou flutuaria sobre a rampa.

CICLO DE EXECUÇÃO

Fluxograma resumido do programa



Menu

ENTER inicia o jogo · TAB abre o editor

Editor

Raycast do mouse: colocar, editar, remover

Jogando

Física: gravidade, colisão, pulo, checkpoints

Pausa / Fim

ESC pausa · TAB volta ao editor

Renderização a cada quadro

`Shadow pass → cena principal (Blinn-Phong + textura + sombra) → HUD`

↻ swapBuffers + pollEvents encerram o quadro e o programa volta ao topo do loop principal.

OBRIGADO PELA ATENÇÃO!